Assignment 4 Report

Harman Bains

March 30, 2026

Course/Assignment # : CMPT 276 D300, Assignment 4

1 Code Smells Identified

My reviewer was Jayden who identified 5 code smells during the code reviewing. The first was a feature envy in static enemy Java file. This was regarding the on collision method in static enemy which was reliant on games data and methods in order to conduct arithmetic in its own class. The second was a missing initialization of a dependency in static enemy Java file. This was regarding the variable of type game called game that was not initialized in the constructor which a null exception would result if on collision method called for the variable. The third smell code found in static enemy was improper naming in the Java comments. This is a small issue such as typos and grammar that needed cleaning up. The fourth code smell was dead code in the cell Java file regarding the set position method. It was noted to not contributing as this was also evident in cell test also for the position test method. The last smell code was missing immutability in the cell Java file. This was regarding the cell type variable being initialized at first.

2 Steps and Re-factoring

There were multiple steps taken to ensure the code smells were addressed. The simplest one conducted first was the improper naming in static enemy which is found through reading within the Java doc comments regarding the particular typo. The reviewer found collision to be misspelled, so as a result two instances were found and verified through target folder of the html showing proof the modified Java doc comment changes. Unfortunately the IDE was not able to identify the typo hence having to read through the Java file. The next code smell was dead code through set position in the cell Java file which was removed. This meant that during the cell test there was a variable to set the position rather than accessing the set position method through cell. It went through successfully as it was in order to assert true of the get position method. The third code smell solved was a missing immutability in the cell Java file that was taking into consideration the cell type variable. These steps required to make it final in order to avoid reassignment of an initialized variable that also did not have any impact on the cell test after the change. The follow up step which again did not have an impact on the testing part with particularly the static enemy and its subclasses was adding the missing initialized dependency of the variable game. The change was done within the static enemy Java file of its own method that set the game variable to null. The reason for not adding game as a third parameter to the static enemy method was there was too much conflict in static enemies and its subclasses along with the testing that needed to be resolved. Furthermore, it becomes contradicted in game map regarding static enemies subclasses thorns and bear trap which would then have to take into consideration game map instead within that class since game is not defined inside. Initializing the game to null ensured a minimal adjustment that on collision would apply as long as the set game method activated the game variable. The last big part left for the single code smell was the feature

envy of Java files between the static enemy and the game. This code smell was the most difficult overall which the feature envy of static enemy towards game required the first measure to be getting the arithmetic conducted in the game Java file itself. What was done is take the logic in the on collision method originally inside the static enemy Java file and then apply it with a slight change to accommodating being within the game Java file creating a different method called static enemy damage. This meant that the on collision method would be modified so if the game was set and not null then the explorer would take damage by accessing the game of the static enemy damage method. The final half left of the code smell was the testing part that specifically looked at static enemy where two tests failed inside after the modifications. The issue was resolved where all the tests passed was eventually adding the parameter to the static enemy damage method of a explorer. The reason is that on collision had a parameter of an explorer and static enemy damage method did not, so originally it was referring to two different explorers which meant damage was being applied differently. Consequently, this ensured the explorer referred in the code was the same.

3 Code Smells Found For Team-mate

There were 4 smell codes Identified in my team-mate who is Nathaniel. The first three were in the chest Java file that first involved dead code. There was an unchecked suppressed warning applied for a method that is being accessed in the testing phase so its role was being contradicted and not needed. Another code smell was an uncommunicative name. The method attempt opening method is was not clear, and could be done better especially for becoming apparent that it is a boolean method since it is describing an action not a state. The last smell code in the chest Java file was structured code needing improvement. The attempt opening method currently used two If conditions and two false conditions which could be changed into a more efficient structure of code. There were next Java file looked into was losing by coins that had a code smell of redundant comments. There was in-line comments for the getters setters repeatedly that could be altered to a more simple quality comment than quantity in this case.